DEEP LEARNING TRANSFORMER SEQ2SEQ

Sequence-to-Sequence Architectures & The Attention Mechanism

By Uditya Narayan Tiwari June 2026

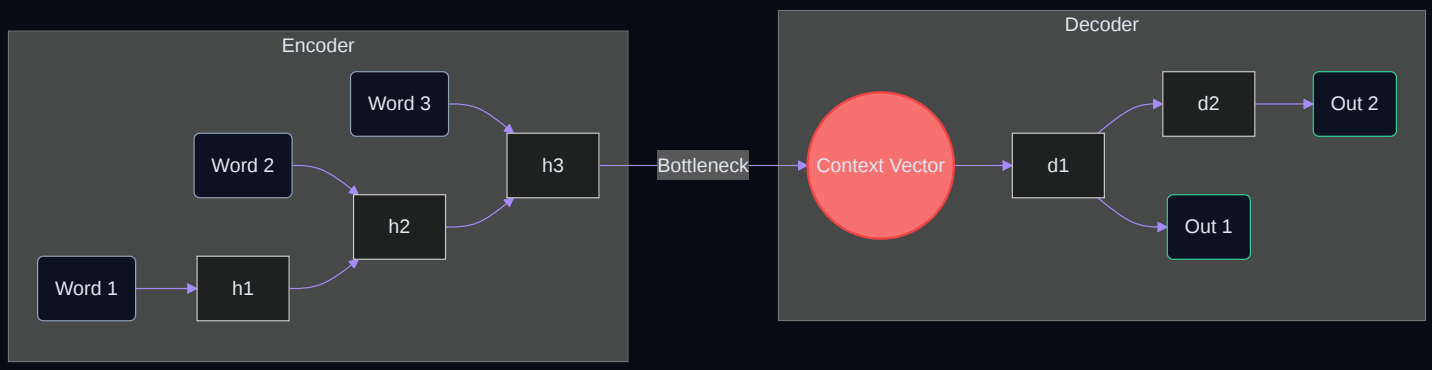
1. Introduction

The sequence-to-sequence (Seq2Seq) model is a landmark architecture in natural language processing. Originally developed for neural machine translation, it converts an input sequence of tokens (such as a sentence in French) into an output sequence of tokens (its translation in English). The foundation of modern language translation, text summarization, and dialogue conversational models is built upon this concept.

2. The Encoder-Decoder Bottleneck

Traditional Seq2Seq networks utilize an **Encoder-Decoder** structure. The encoder (an RNN/LSTM) processes the source sentence token-by-token, updating its recurrent state. The final hidden state vector acts as the **context vector**, which is intended to compress the semantics of the entire sequence.

TRADITIONAL SEQ2SEQ BOTTLENECK (.MMD)



This model suffers from a critical theoretical flaw: **the bottleneck issue**. All information, regardless of input length (whether 5 words or 100 words), must be compressed into the same fixed-size vector space. When sentences grow longer, recurrent units tend to "forget" earlier tokens due to vanishing gradients during backpropagation.

The Mathematics of Recurrent Forgetfulness

In standard Recurrent Neural Networks (RNNs), the hidden state h_t is computed as:

ENCODER RECURRENT EQUATION

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

During backpropagation through time (BPTT), the gradient of the loss at step T with respect to the hidden state at step t is calculated via chain-rule matrix products:

BPTT GRADIENT CHAIN RULE

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

Since the Jacobian $\partial h_k / \partial h_{k-1} = \text{diag}(1 - \tanh^2(\dots)) \cdot W_{hh}^T$, multiplying this matrix repeatedly over $T - t$ steps causes the gradient values to either vanish to zero (if the eigenvalues of W_{hh} are less than 1) or explode (if greater than 1). Consequently, early words x_1, x_2 lose representation capacity in the final state h_T.

3. Intuition Behind Attention

Attention solves the bottleneck by exposing all encoder hidden states h_j directly to the decoder. At each step t of decoding, instead of reading a single context vector, the decoder queries all the encoder hidden states. It determines which states are most relevant to the word it is currently generating, and constructs a custom context vector c_t.

Intuitive Metaphor

Rather than writing a translation entirely from memory after reading the sentence once, the decoder is given a "shortcut lookup". While writing each word, it scans the original source text to check which words to translate next.

4. Mathematical Formulation

Let the encoder hidden state vectors be $h_1, h_2, \ldots, h_{T_x}$ (where T_x is input sequence length). Let s_t represent the hidden state of the decoder at time step t. The context vector c_t is computed as:

CONTEXT VECTOR EQUATION

$$c_t = \sum_{j=1}^{T_x} \alpha_{tj} h_j$$

The attention coefficients (or alignment weights) α_{tj} represent how much focus the output word at position t should give to the input word at position j. These weights are computed by applying a softmax function to the similarity scores:

SOFTMAX NORMALIZATION

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^{T_x} \exp(e_{tk})}$$

Where $e_{tj} = \text{score}(s_{t-1}, h_j)$ measures how well the inputs around position j and the output at position t align.

5. Bahdanau (Additive) vs. Luong (Multiplicative)

There are two prominent formulations of attention within recurrent architectures:

1. Bahdanau (Additive) Attention

Introduced in 2014, this uses a feed-forward layer to learn the similarity score. It computes the score by feeding the decoder state s_{t-1} and encoder state h_j into a non-linear $\tanh$ layer:

BAHDANAU ADDITIVE SCORE

$$\text{score}(s_{t-1}, h_j) = v_a^T \tanh(W_a s_{t-1} + U_a h_j)$$

2. Luong (Multiplicative) Attention

Introduced in 2015, Luong attention simplifies the scoring system. It uses dot-products, which are faster and highly optimized for modern hardware:

LUONG MULTIPLICATIVE SCORE

$$\text{score}(s_t, h_j) = s_t^T W h_j$$

Architectural Differences & State Timing

Beyond mathematical formulations, Bahdanau and Luong attention differ in their **state calculation pipelines**:

- **Bahdanau (Additive) State Order:** Uses the *previous* decoder state s_{t-1} to calculate alignment scores with encoder hidden states h_j. The resulting context vector c_t is then concatenated with the current output embedding and fed into the decoder recurrent cell to calculate the *current* decoder state s_t.
- **Luong (Multiplicative) State Order:** The decoder recurrent cell calculates the current state s_t first. This state is then compared against all encoder states h_j to yield attention weights, compiling the context vector c_t. The vector c_t and s_t are then combined via a projection matrix to create an auxiliary attentional hidden state $\tilde{s}_t = \tanh(W_c[c_t; s_t])$ which is fed to the output classifier.

6. The Query, Key, and Value Metaphor

In Transformer models (Vaswani et al., 2017), recurrence is completely replaced by the Query, Key, and Value (QKV) system. Rather than calculating alignment between recurrent states, each input word vector x is projected into three distinct vectors by learned weight matrices:

- **Query (Q)**: What the word is looking for.
- **Key (K)**: The index label used for matching queries.
- **Value (V)**: The actual semantic payload of the word.

SCALED DOT-PRODUCT ATTENTION

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

In matrix format, given a sequence of input embedding vectors packed as row vectors in matrix $X \in \mathbb{R}^{N \times d_{\text{model}}}$, we compute the projections:

Q, K, V PROJECTIONS

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V$$

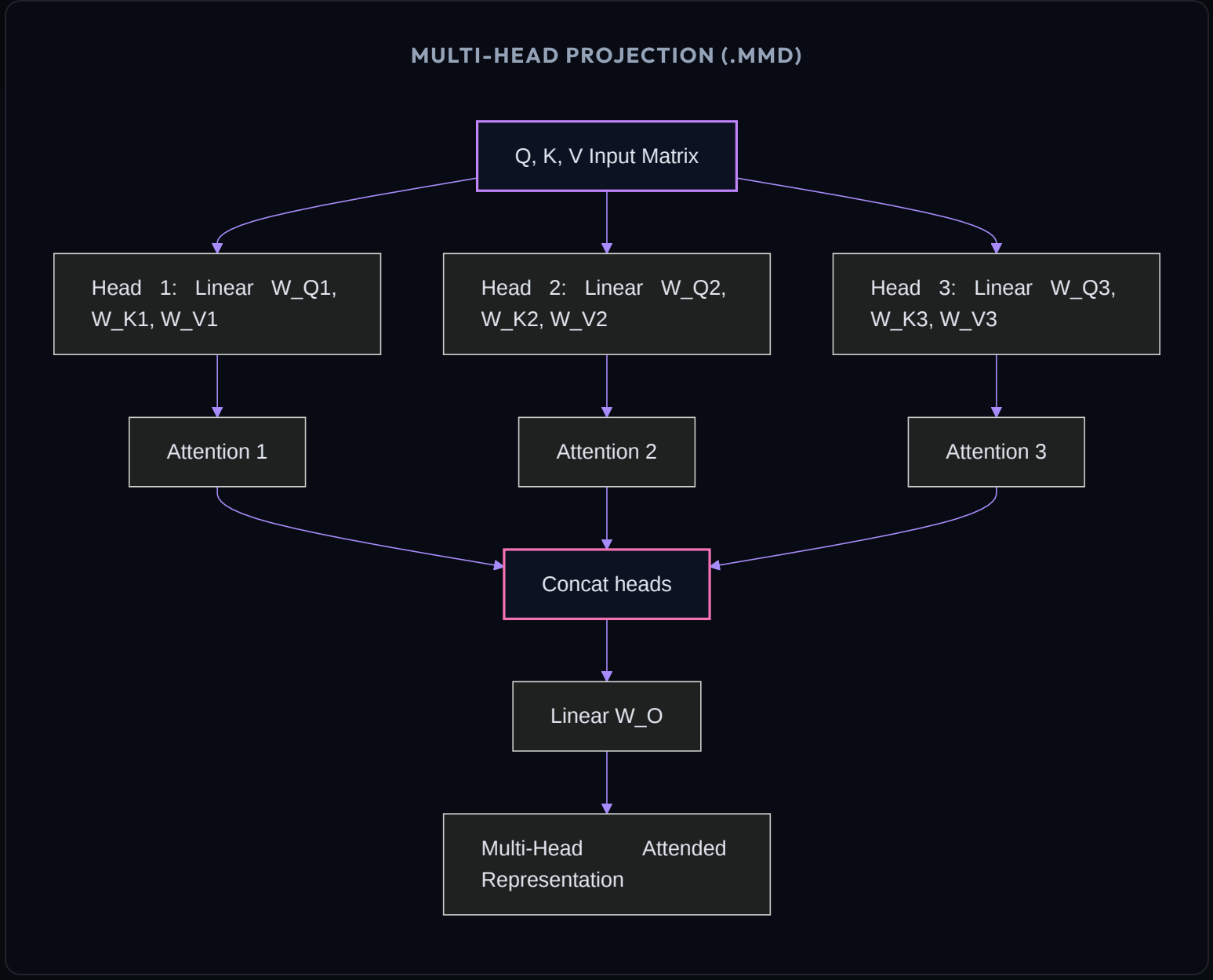
$$\text{where } Q \in \mathbb{R}^{N \times d_k},\ K \in \mathbb{R}^{N \times d_k},\ V \in \mathbb{R}^{N \times d_v}$$

Multiplying $Q \cdot K^T$ generates an $N \times N$ similarity matrix. Each cell (i, j) in this matrix stores the dot-product similarity score between the query of token i and the key of token j. Applying softmax along the rows translates these scores into attention weights.

> **Why divide by √d k ?**
>
> For high dimensionality, the dot product $Q \cdot K^T$ grows large in magnitude. This pushes the softmax function into regions with extremely small gradients. Dividing by the scaling factor $\sqrt{d_k}$ stabilizes training.

7. Multi-Head Attention

Instead of performing a single attention function, Multi-Head Attention projects Queries, Keys, and Values h times in parallel. This allows the model to capture different types of relationships simultaneously (e.g., one head handles subject-verb matching, another handles coreferences, and another handles immediate neighbors).



8. Complexity & Tradeoffs

By shifting from recurrence to fully parallelized self-attention, Transformers achieve massive training scale. However, this comes at a computational cost:

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Recurrent (RNN)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$

While recurrent networks are linear in complexity with respect to sentence length, they must process words sequentially. Self-attention layers compute all relationships in parallel ($O(1)$ operations), but require quadratic complexity ($O(n^2)$) due to the computation of the pairwise attention similarity matrix.

9. Implementation from Scratch

Below is a standard PyTorch implementation of the scaled dot-product attention equation:

```python
import torch
import torch.nn.functional as F

def scaled_dot_product_attention(q, k, v):
    # q size: [batch, heads, seq_len, d_k]
    # k size: [batch, heads, seq_len, d_k]
    # v size: [batch, heads, seq_len, d_v]

    d_k = q.size(-1)

    # 1. Similarity Matrix
    scores = torch.matmul(q, k.transpose(-2, -1))

    # 2. Scaling
    scaled_scores = scores / (d_k ** 0.5)

    # 3. Softmax Distribution
    attention_weights = F.softmax(scaled_scores, dim=-1)

    # 4. Values Weighted Combination
    output = torch.matmul(attention_weights, v)

    return output, attention_weights
```